

Day-13

DATE:25-08-25

NAME: NANCY M

[BookController.cs:](#)

```
using Day12.Models;
using Day12Api.Models;
using Day12Api.Service;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace Day12Api.Controllers
{
    [ApiController]
    [Route("[controller]")]
    public class BookController : Controller
    {
        private readonly IBookService _bookService;

        public BookController(IBookService bookService)
        {
            _bookService = bookService;
        }

        [HttpGet("GetAllBooks")]
        public IActionResult GetAllBooks() => Ok(_bookService.GetAllBooks());

        [HttpPost("AddAuthor")]
        public IActionResult AddAuthor(Author author) => Ok(_bookService.AddAuthor(author));

        [HttpPost("AddNewBook")]
        public IActionResult AddNewBook(BookAndAuthorEntry entry) =>
Ok(_bookService.AddNewBook(entry));

        [HttpPost("FetchAllNewBooks")]
        public IActionResult FetchAllNewBooks() => Ok(_bookService.FetchAllNewBooks());

        [HttpGet("FetchBookByAuthor")]
        public IActionResult FetchBookByAuthor(string authorName) =>
Ok(_bookService.FetchBookByAuthor(authorName));

        [HttpGet("FetchBookByAuthorProjection")]
    }
```

```

    public IActionResult FetchBookByAuthorProjection(string authorName) =>
    Ok(_bookService.FetchBookByAuthorProjection(authorName));

    [HttpGet("FetchSalesByAuthor")]
    public IActionResult FetchSalesByAuthor(string authorName) =>
    Ok(_bookService.FetchSalesByAuthor(authorName));

    [HttpGet("FetchProfitByAuthor")]
    public IActionResult FetchProfitByAuthor(string authorName) =>
    Ok(_bookService.FetchProfitByAuthor(authorName));

    [HttpGet("FetchAllDetails")]
    public IActionResult FetchAllDetails(string authorname) =>
    Ok(_bookService.FetchAllDetails(authorname));

    [HttpGet("FetchAllBookByNP")]
    public IActionResult FetchAllBookByNP(string authorName, int bookPrice) =>
    Ok(_bookService.FetchAllBookByNP(authorName, bookPrice));

    [HttpGet("FetchAllUserAdmin")]
    public IActionResult FetchAllUserAdmin() => Ok(_bookService.FetchAllUserAdmin());

    [HttpGet("FetchAverage")]
    public IActionResult FetchAverage() => Ok(_bookService.FetchAverage());

    [HttpGet("GetByYear")]
    public IActionResult GetByYear(int year) => Ok(_bookService.GetByYear(year));

    [HttpGet("FetchBt200500")]
    public IActionResult FetchBt200500() => Ok(_bookService.FetchBt200500());

    [HttpPost("InsertSales")]
    public IActionResult InsertSales(SalesEntry entry) =>
    Ok(_bookService.AddSalesInfo(entry));

    [Authorize]
    [HttpPost("InsertBook")]
    public IActionResult InsertBook(Book book) => Ok(_bookService.AddBook(book));

    [HttpGet("ViewBooks")]
    public IActionResult GetAllBook() => Ok(_bookService.GetAllBook());

    [Authorize(Roles = "Admin")]
    [HttpDelete("RemoveBook")]

```

```

        public IActionResult DeleteBook(int id) => Ok(_bookService.DeleteBook(id));

        [HttpPost("GetBooksOrderedByPrice")]
        public IActionResult GetBooksOrderedByPrice([FromQuery] bool ascending) =>
        Ok(_bookService.GetBooksOrderedByPrice(ascending));

        [HttpGet("GetTop5Books")]
        public IActionResult GetTop5Books() => Ok(_bookService.GetTop5Books());

        [HttpGet("GetCostDetails")]
        public IActionResult GetCostDetails() => Ok(_bookService.GetCostDetails());
    }
}

```

[IBookService.cs:](#)

```

using Day12.Models;
using Day12Api.Models;

namespace Day12Api.Service
{
    public interface IBookService
    {
        IEnumerable<Book> GetAllBooks();
        string AddAuthor(Author author);
        string AddNewBook(BookAndAuthorEntry bookAndAuthorEntry);
        IEnumerable<NewBook> FetchAllNewBooks();
        IEnumerable<NewBook> FetchBookByAuthor(string authorName);
        IEnumerable<object> FetchBookByAuthorProjection(string authorName);
        object FetchSalesByAuthor(string authorName);
        IEnumerable<object> FetchProfitByAuthor(string authorName);
        IEnumerable<object> FetchAllDetails(string authorName);
        IEnumerable<object> FetchAllBookByNP(string authorName, int bookPrice);
        IEnumerable<User> FetchAllUserAdmin();
        double FetchAverage();
        IEnumerable<object> GetByYear(int year);
        IEnumerable<object> FetchBt200500();
        string AddSalesInfo(SalesEntry entry);
        string AddBook(Book book);
        IEnumerable<Book> GetAllBook();
        string DeleteBook(int id);
        IEnumerable<Book> GetBooksOrderedByPrice(bool ascending);
        IEnumerable<Book> GetTop5Books();
        object GetCostDetails();
    }
}

```

```
}  
}
```

[BookService.cs](#)

```
using Day12.Models;  
using Day12Api.Context;  
using Day12Api.Models;  
using Microsoft.EntityFrameworkCore;  
  
namespace Day12Api.Service  
{  
    public class BookService : IBookService  
    {  
        private readonly MyAppDbContext _context;  
  
        public BookService(MyAppDbContext context)  
        {  
            _context = context;  
        }  
  
        public IEnumerable<Book> GetAllBooks()  
        {  
            return new List<Book>  
            {  
                new Book { Title = "stories", Description = "kids fav", Rating = 4 },  
                new Book { Title = "novel", Description = "adult fav", Rating = 3 }  
            };  
        }  
  
        public string AddAuthor(Author author)  
        {  
            _context.Authors.Add(author);  
            _context.SaveChanges();  
            return "author added";  
        }  
  
        public string AddNewBook(BookAndAuthorEntry bookAndAuthorEntry)  
        {  
            var author = _context.Authors.FirstOrDefault(x => x.AuthorName ==  
bookAndAuthorEntry.AuthorName);  
            if (author == null) return "invalid author";  
  
            var newBook = new NewBook
```

```

    {
        Title = bookAndAuthorEntry.BookTitle,
        Price = bookAndAuthorEntry.Price,
        Sales = bookAndAuthorEntry.Sales,
        AuthorId = author.AuthorId
    };

    _context.NewBooks.Add(newBook);
    _context.SaveChanges();
    return "added the book";
}

public IEnumerable<NewBook> FetchAllNewBooks()
{
    return _context.NewBooks.Include(x => x.author).ToList();
}

public IEnumerable<NewBook> FetchBookByAuthor(string authorName)
{
    return _context.NewBooks.Include(x => x.author)
        .Where(x => x.author.AuthorName == authorName)
        .ToList();
}

public IEnumerable<object> FetchBookByAuthorProjection(string authorName)
{
    return _context.NewBooks
        .Where(x => x.author.AuthorName == authorName)
        .Select(y => new { y.Title, Rate = y.Price, Author = y.author.AuthorName })
        .ToList();
}

public object FetchSalesByAuthor(string authorName)
{
    var total = _context.NewBooks.Where(x => x.author.AuthorName == authorName)
        .Sum(y => y.Sales);

    return new { totalsales = total, Authorname = authorName };
}

public IEnumerable<object> FetchProfitByAuthor(string authorName)
{
    return _context.NewBooks.Where(x => x.author.AuthorName == authorName)

```

```

        .Select(y => new { y.Title, Rate = y.Price, Profit = y.Sales * y.Price, Author =
y.author.AuthorName })
        .ToList();
    }

    public IEnumerable<object> FetchAllDetails(string authorName)
    {
        return _context.TotalSales.Where(x => x.Book.author.AuthorName == authorName)
            .Select(x => new
            {
                Bookname = x.Book.Title,
                BookPrice = x.Book.Price,
                Booksales = x.Book.Sales,
                bookcopies = x.No_of_copies,
                bookyear = x.year,
            });
    }

    public IEnumerable<object> FetchAllBookByNP(string authorName, int bookPrice)
    {
        return _context.NewBooks.Where(x => x.author.AuthorName == authorName && x.Price
== bookPrice)
            .Select(z => new { Bookname = z.Title });
    }

    public IEnumerable<User> FetchAllUserAdmin()
    {
        return _context.Users.Where(x => x.Role == "admin").ToList();
    }

    public double FetchAverage()
    {
        return _context.Books.Average(x => x.Rating);
    }

    public IEnumerable<object> GetByYear(int year)
    {
        return _context.TotalSales.Where(x => x.year == year)
            .Select(y => new
            {
                BookName = y.Book.Title,
                BookTitle = y.Book.Title,
                BookPrice = y.Book.Price,
                Bookauthor = y.Book.author.AuthorName,
            });
    }

```

```

    });
}

public IEnumerable<object> FetchBt200500()
{
    return _context.NewBooks.Where(x => x.Price >= 200 && x.Price <= 500)
        .Select(y => new { BookName = y.Title, BookAuthor = y.author.AuthorName });
}

public string AddSalesInfo(SalesEntry entry)
{
    var book = _context.NewBooks.FirstOrDefault(x => x.Title == entry.Bookname);
    if (book == null) return "book not found";

    Sales sales = new Sales
    {
        BookId = book.NewBookId,
        No_of_copies = entry.Num_of_copies,
        year = entry.Year,
    };

    _context.TotalSales.Add(sales);
    _context.SaveChanges();
    return "sales added";
}

public string AddBook(Book book)
{
    _context.Books.Add(book);
    _context.SaveChanges();
    return "added the book to the store";
}

public IEnumerable<Book> GetAllBook()
{
    return _context.Books.ToList();
}

public string DeleteBook(int id)
{
    var book = _context.Books.FirstOrDefault(x => x.Id == id);
    if (book == null) return "book not found";

    _context.Books.Remove(book);
}

```

```

        _context.SaveChanges();
        return "Deleted";
    }

    public IEnumerable<Book> GetBooksOrderedByPrice(bool ascending)
    {
        return ascending
            ? _context.Books.OrderBy(b => b.Price).ToList()
            : _context.Books.OrderByDescending(b => b.Price).ToList();
    }

    public IEnumerable<Book> GetTop5Books()
    {
        return _context.Books.OrderByDescending(b => b.Price)
            .Take(5)
            .ToList();
    }

    public object GetCostDetails()
    {
        var total = _context.Books.Sum(b => b.Price);
        var average = _context.Books.Average(b => b.Price);
        return new { TotalPrice = total, AveragePrice = average };
    }
}

```


localhost:5246/swagger/index.html

Day 12 API

http://localhost:5246/swagger/v1/swagger.json

Book

GET /Book/GetAllBooks

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5246/Book/GetAllBooks' \
  -H 'accept: */*'
```

Request URL

http://localhost:5246/Book/GetAllBooks

Server response

localhost:5246/swagger/index.html

Request URL

http://localhost:5246/Book/GetAllBooks

Server response

Code	Details
200	Response body <pre>[{ "id": 0, "title": "stories", "description": "kids fav", "rating": 4, "price": 0 }, { "id": 0, "title": "novel", "description": "adult fav", "rating": 3, "price": 0 }]</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Tue, 26 Aug 2025 08:08:56 GMT server: Kestrel transfer-encoding: chunked</pre>

Responses

Code	Description
200	OK

Parameters

Cancel

Name	Description
authorName	
string	Robert
(query)	

Execute

Clear

Responses

Curl

curl -X 'GET' \nhttp://localhost:5246/Book/FetchBookByAuthor?authorName=Robert' \n-H 'accept: */*' \n

Request URL

http://localhost:5246/Book/FetchBookByAuthor?authorName=Robert

Server response

Code	Details
200	

Server response

Code	Details
200	Response body <pre>[{ "newBookId": 3, "title": "Clean Code", "price": 59, "sales": 9, "authorId": 2, "author": { "authorId": 2, "authorName": "Robert" } }, { "newBookId": 9, "title": "Conclave", "price": 70, "sales": 10, "authorId": 2, "author": { "authorId": 2, "authorName": "Robert" } }]</pre> Download Response headers content-type: application/json; charset=utf-8 date: Tue, 26 Aug 2025 08:10:18 GMT server: Kestrel transfer-encoding: chunked

Responses

Code	Description	Links
------	-------------	-------